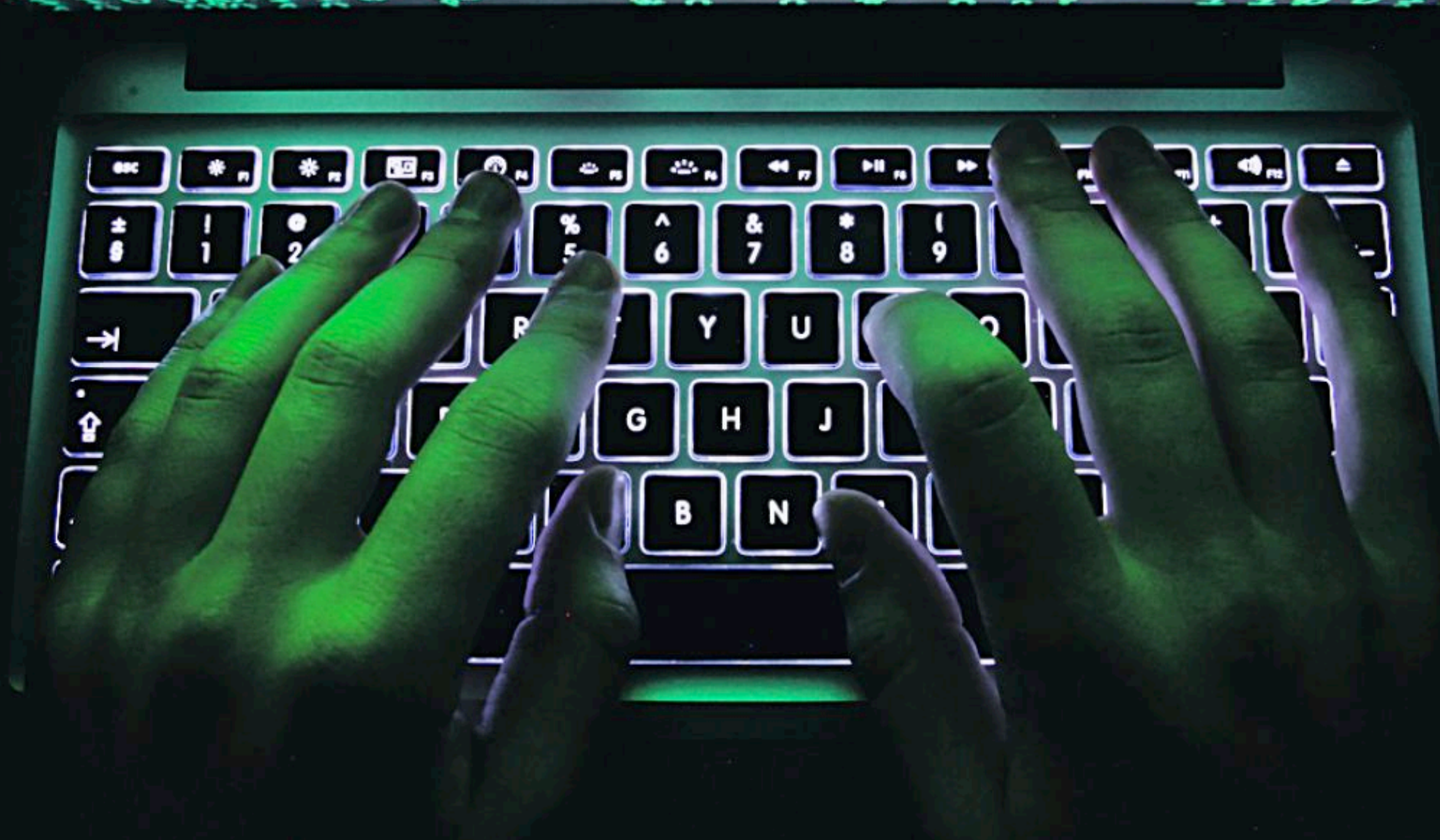


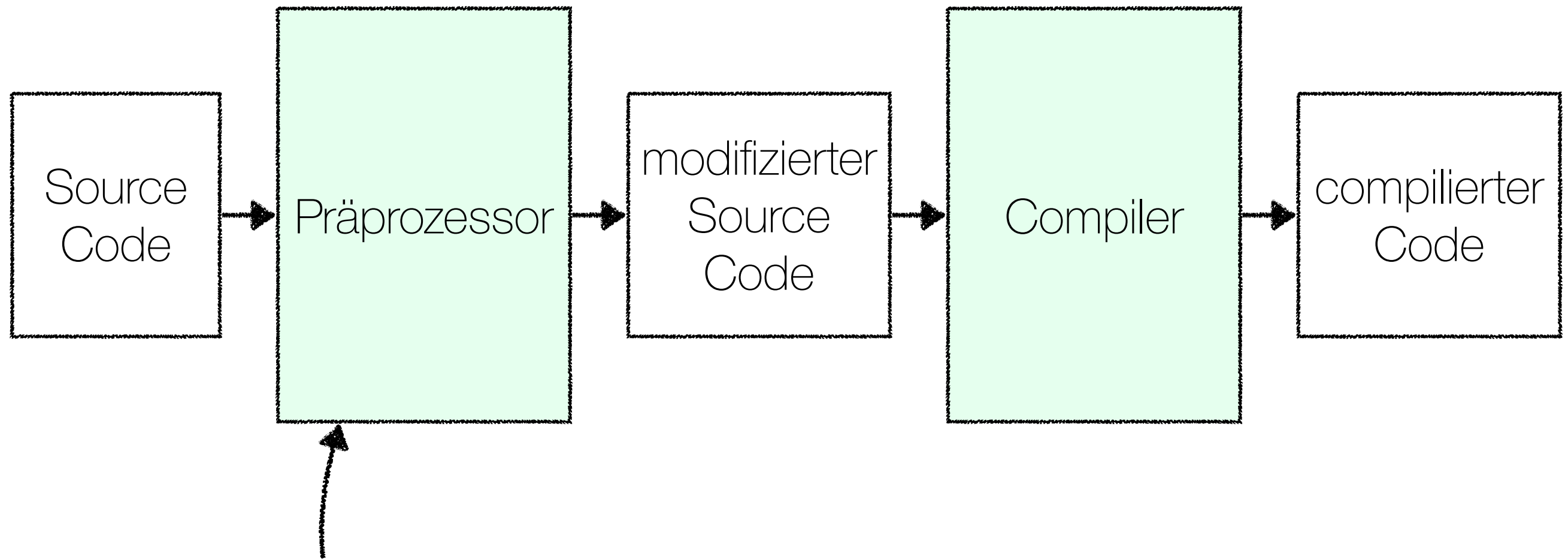
PROGRAMMIERUNG 2

Kapitel 09: C - Programmstruktur



Prof. Dr. Markus Esch
htw saar

Präprozessor Direktiven



- ▶ Präprozessor Direktiven auswerten
- ▶ Code-Ersetzungen

Präprozessor-Direktiven

Befehl	Bedeutung
<code>#include</code>	Fügt Text aus einer anderen Quelltextdatei ein.
<code>#define</code>	Definiert ein Makro.
<code>#undef</code>	Entfernt ein Makro.
<code>#if</code>	Bedingte Compilierung in Abhängigkeit der nachstehenden Bedingung.
<code>#ifdef</code>	Bedingte Compilierung in Abhängigkeit, ob ein Makroname definiert ist.
<code>#ifndef</code>	Bedingte Compilierung in Abhängigkeit, ob ein Makroname nicht definiert ist.
<code>#else</code>	Alternative Compilierung, wenn die Bedingung des vorstehenden <code>#if</code> , <code>#elif</code> , <code>#ifdef</code> oder <code>#ifndef</code> nicht erfüllt ist.
<code>#endif</code>	Beendet den Block mit der bedingten Compilierung.
<code>#line</code>	Liefert eine Zeilennummer für Compiler-Meldungen.
<code>defined</code>	Liefert eine 1, wenn der nachstehende Makroname definiert ist, sonst eine 0.
<code>#operator</code>	Ersetzt in einen Makroparameter durch eine konstante Zeichenkette.
<code>#pragma</code>	Gibt Compiler- und System-abhängige Informationen an den Compiler.
<code>#error</code>	Erzeugt einen Fehler während der Compilierung mit der angegebenen Fehlermeldung.

#include

Zeile wird durch
Quelltext der Datei
stdio.h ersetzt

< . . . >

suche im Standard-
Include-Pfad des
Compilers

```
#include <stdio.h>
#include "myheader.h"

/* A simple C program */

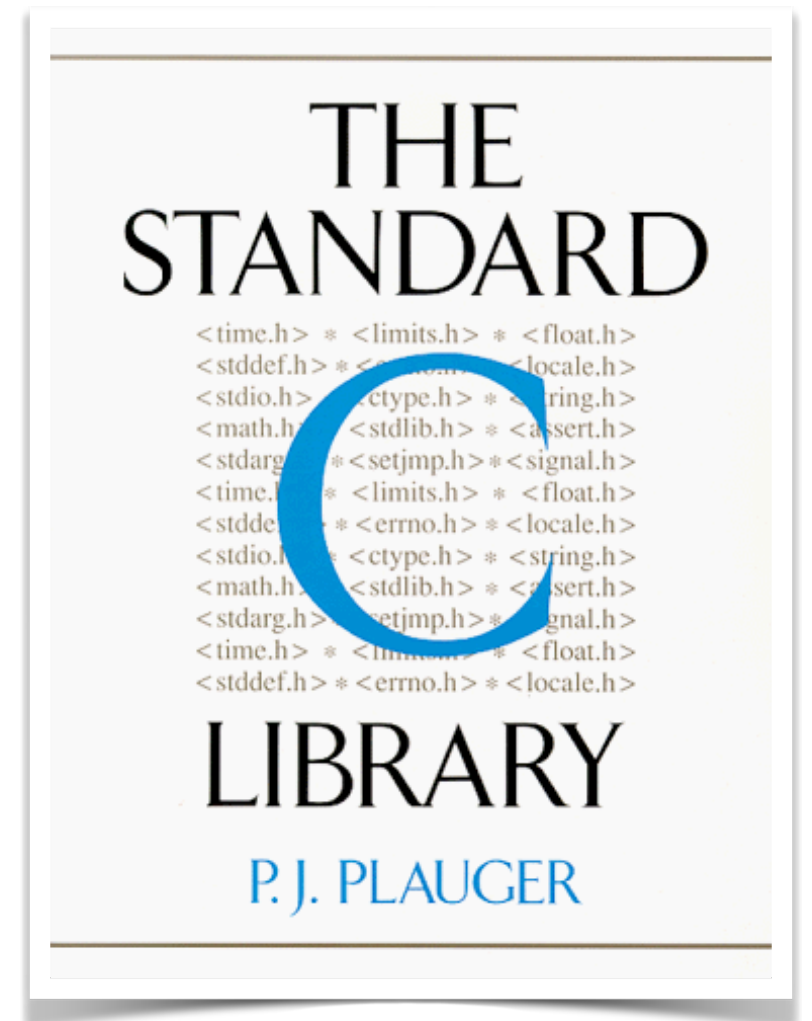
int main ()
{
    printf("Hello World!\n");
}
```

" . . . "

suche im aktuellen
Verzeichnis

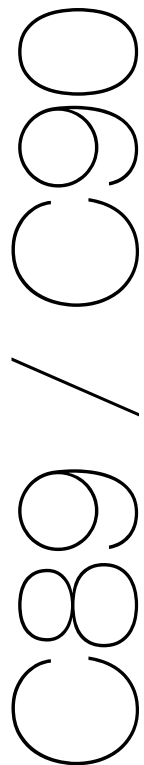
C Standard **Library**

- ▶ Standardbibliothek von C
 - ▶ ~**200** gängige Funktionen für:
 - ▶ Ein- und Ausgabe
 - ▶ mathematische Operationen
 - ▶ Verarbeitung von Zeichenketten
 - ▶ Speicherverwaltung
 - ▶ etc.
- ▶ Standardkonforme C-Implementierung muss Library bereitstellen
 - ▶ meist vom Compilerhersteller implementiert
 - ▶ z.B.: **glibc** (GNU-C-Bibliothek)



C Standard **Library**

Header-Datei	Funktion
<code>assert.h</code>	Assertions
<code>ctype.h</code>	Tests auf bestimmte Zeichentypen
<code>errno.h</code>	Codes von Systemfehlern
<code>float.h</code>	Angaben zu den Wertbereichen von Gleitkommazahlen
<code>limits.h</code>	Angaben zu Beschränkungen des verwendeten Systems
<code>locale.h</code>	Einstellungen des Gebietsschemas
<code>math.h</code>	mathematische Funktionen
<code>setjmp.h</code>	erweiterte Sprungfunktionen
<code>signal.h</code>	Signalbehandlung
<code>stdarg.h</code>	Argumentbehandlung für variadische Funktionen
<code>stddef.h</code>	zusätzliche Typdefinitionen
<code>stdio.h</code>	Ein- und Ausgabe
<code>stdlib.h</code>	vermischte Standardfunktionen, u. a. Speicherverwaltung
<code>string.h</code>	Zeichenkettenoperationen
<code>time.h</code>	Datum und Uhrzeit



C Standard **Library**

Header-Datei	Funktion
iso646.h	alternative Schreibweisen für logische und bitweise Operatoren
wchar.h	Unterstützung für Unicode-Zeichen
wctype.h	wie ctype.h, für Unicode-Zeichen
complex.h	Komplexe Zahlen
fenv.h	Einstellungen für das Rechnen mit Gleitkommazahlen
inttypes.h	Konvertierung und Formatierung für erweiterte Ganzzahltypen
stdbool.h	Unterstützung für Boolesche Variablen
stdint.h	plattformunabhängige Definition von Ganzzahltypen
tgmath.h	typgenerische Makros für mathematische Funktionen
stdalign.h	Makros für die Speicherausrichtung von Objekten
stdatomic.h	Typen und Makros für atomare Operationen zwischen Threads
stdnoreturn.h	Definition des Noreturn-Makros
threads.h	Unterstützung für Threads, Mutexes und Monitore
uchar.h	Unterstützung für UTF-16 und UTF-32

C95 / C99 / C11

Definition einer
Zeichenkette, die vom
Präprozessor ersetzt wird

#define

symbolische
Konstante

```
#include <stdio.h>
#define PI 3.1415926f

int main ()
{
    float A, C, d;
    printf("Enter circle diameter: ");
    scanf("%f", &d);
    C = d * PI;
    A = d * d * PI / 4;
    printf("Area: %f; Circumference %f\n", A, C);
}
```

nach
Präprozessor-
Ersetzung

```
C = d * 3.1415926f;
A = d * d * 3.1415926f / 4;
```

#define

```
#include <stdio.h>
#define GANZZAHL      int
#define SCHREIB      printf(
#define END          );
#define EINGABE      scanf(
#define ENDESTART    return 0;
#define NEUEZEILE    printf("\n");
#define START        int main()
#define BLOCKANFANG  {
#define BLOCKENDE    }

START
BLOCKANFANG
    GANZZAHL zahl;
    SCHREIB "Hallo Welt" END
    NEUEZEILE
    SCHREIB "Zahleingabe: " END
    EINGABE "%d", &zahl END
    SCHREIB "Die Zahl war %d\n", zahl END
ENDESTART
BLOCKENDE
```

```
int main()
{
    int zahl;
    printf( "Hallo Welt" );
    printf("\n");
    printf( "Zahleingabe: " );
    scanf( "%d", &zahl );
    printf( "Die Zahl war %d\n", zahl );
    return 0;
}
```

Präprozessor



#define Makros

```
#include <stdio.h>
#include <stdlib.h>
#define SMALLER_100(x) ((x) < 100)
```

```
int main(void) {
    int i;
    printf("Enter a number\n");
    scanf("%d", &i);
    if(SMALLER_100(i))
        printf("Your number is < 100 \n");
    else
        printf("Your number is > 100\n");
    return 0;
}
```

Makro-
Programm

Präprozessor
- Ersetzung

```
if((i)<100)
```

#define Makros: **Beispiel**

```
#include <stdio.h>
#include <stdlib.h>
#define SWAP(x,y)    { \
    int j; \
    j=x; x=y; y=j; \
}

int main(void) {
    int a;
    int b;
    printf("Enter a number\n");
    scanf("%d", &a);
    printf("Enter a number\n");
    scanf("%d", &b);
    printf("You entered a: %d, b:%d\n", a, b);
    SWAP(a,b)
    printf("Swaped: a: %d, b:%d\n", a, b);
    return 0;
}
```

Bedingte Compilierung

```
#include<stdio.h>
#define DEBUG
```

```
int main() {
    int a=2, b=3, r;
    r = (2*a) + (2*b);
```

```
#ifdef DEBUG
```

```
printf("* Debug: result = (2*%d) + (2*%d);\n", a, b);
```

```
#else
```

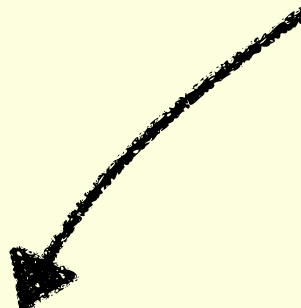
```
printf("The result is: %d\n", r);
```

```
#endif
```

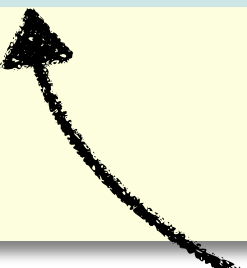
```
return 0;
```

```
}
```

wird nur compiliert wenn
symbolische Konstante
definiert ist



wird nur compiliert wenn
symbolische Konstante
NICHT definiert ist



Bedingte Compilierung

```
#include<stdio.h>
#define DEBUG 1
int main() {
    int a=2, b=3, r;
    r = (2*a) + (2*b);
    #if DEBUG >= 1
    printf("* Debug: result = (2*%d) + (2*%d);\n", a, b);
    #endif
    #if DEBUG >= 2
    printf("* Debug: a=%d, b=%d, result=%d\n", a, b, r);
    #endif
    #ifndef DEBUG
    printf("The result is: %d\n", r);
    #endif
    return 0;
}
```

Prüfung auf einen bestimmten Wert



wird nur compiliert wenn
symbolische Konstante
NICHT definiert ist



Verwendung von **Macros**

- ▶ plattformspezifischer Code

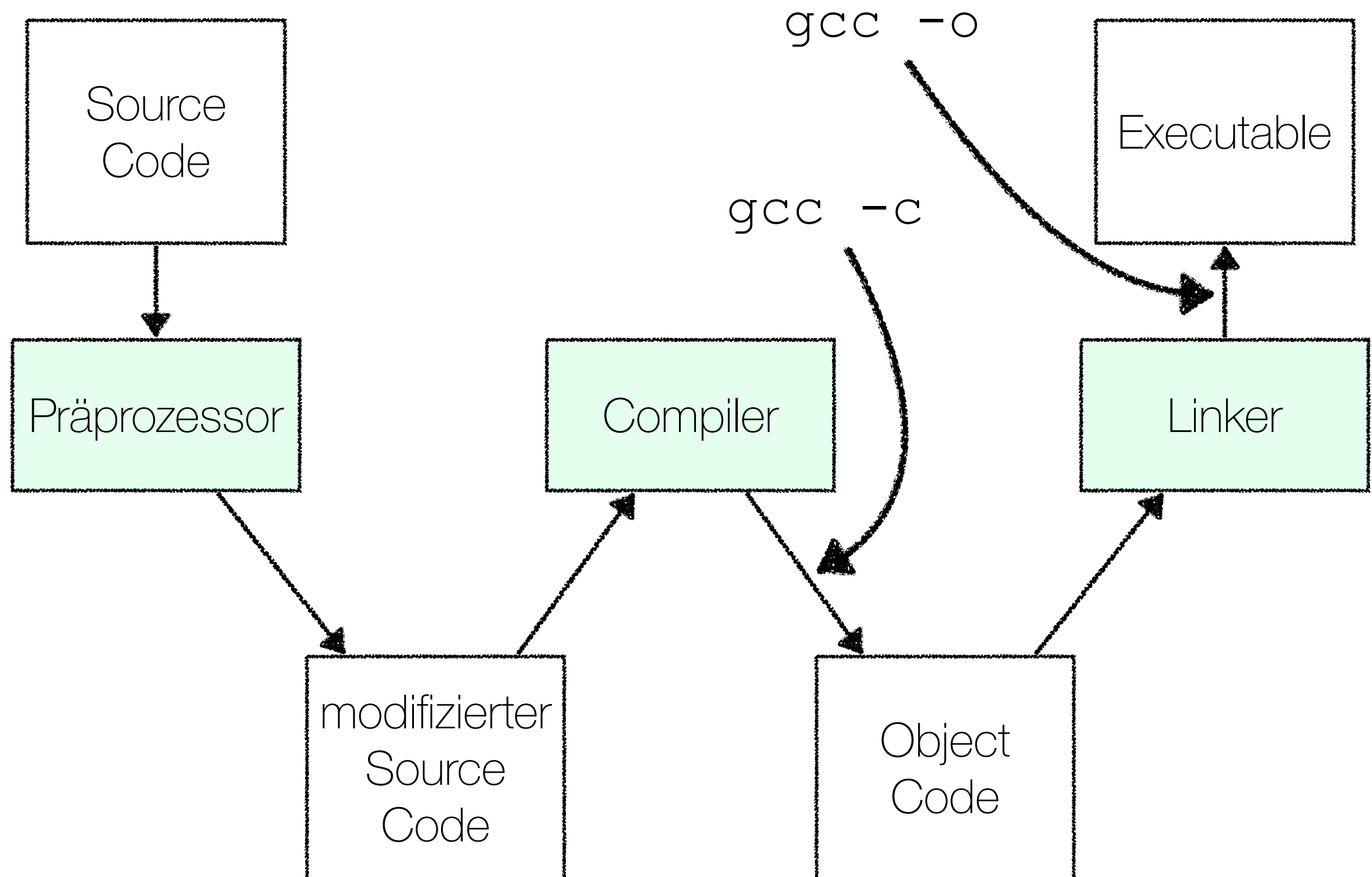
```
#ifdef _WIN32
    #define PATH_SEPARATOR '\\\'
#else
    #define PATH_SEPARATOR '/'
#endif
```

- ▶ Konstanten

```
#define MAX_SIZE 1024
```

- ▶ bedingte Compilierung
- ▶ ifdef-Guards bzw. Include-Guards
 - ▶ später
- ▶ möglichste **KEINE** komplexe Logik

Linker

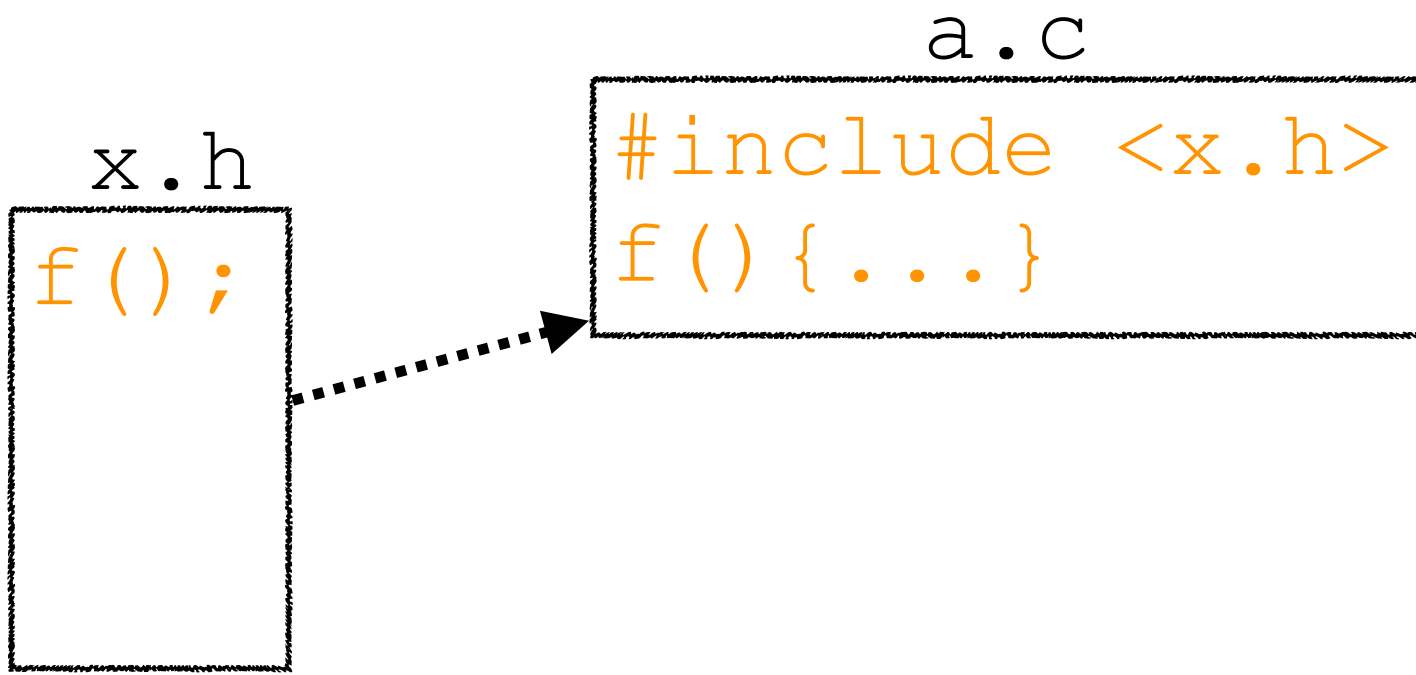


Programm**struktur**

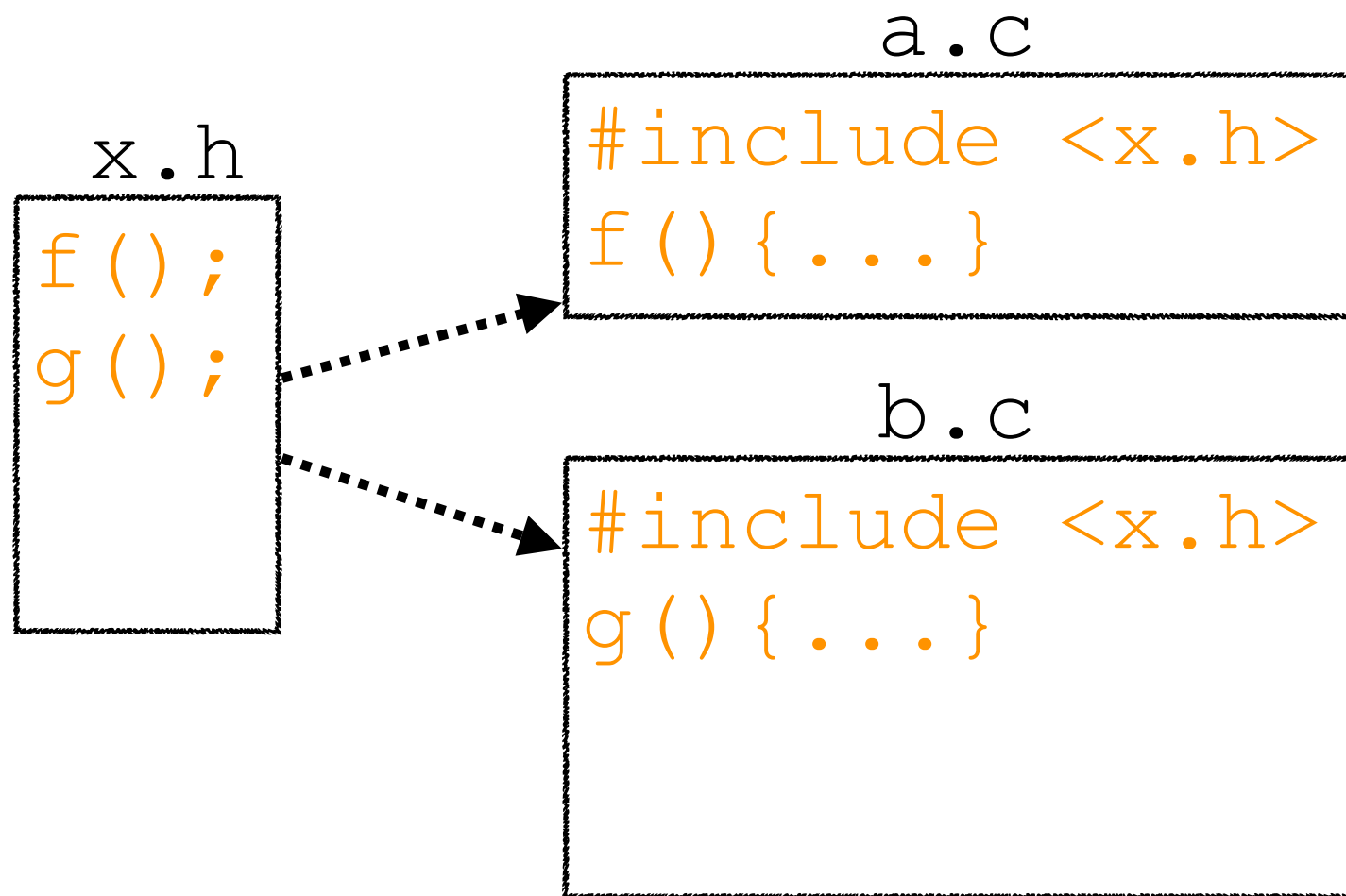
a.c

```
f () { . . . }
```

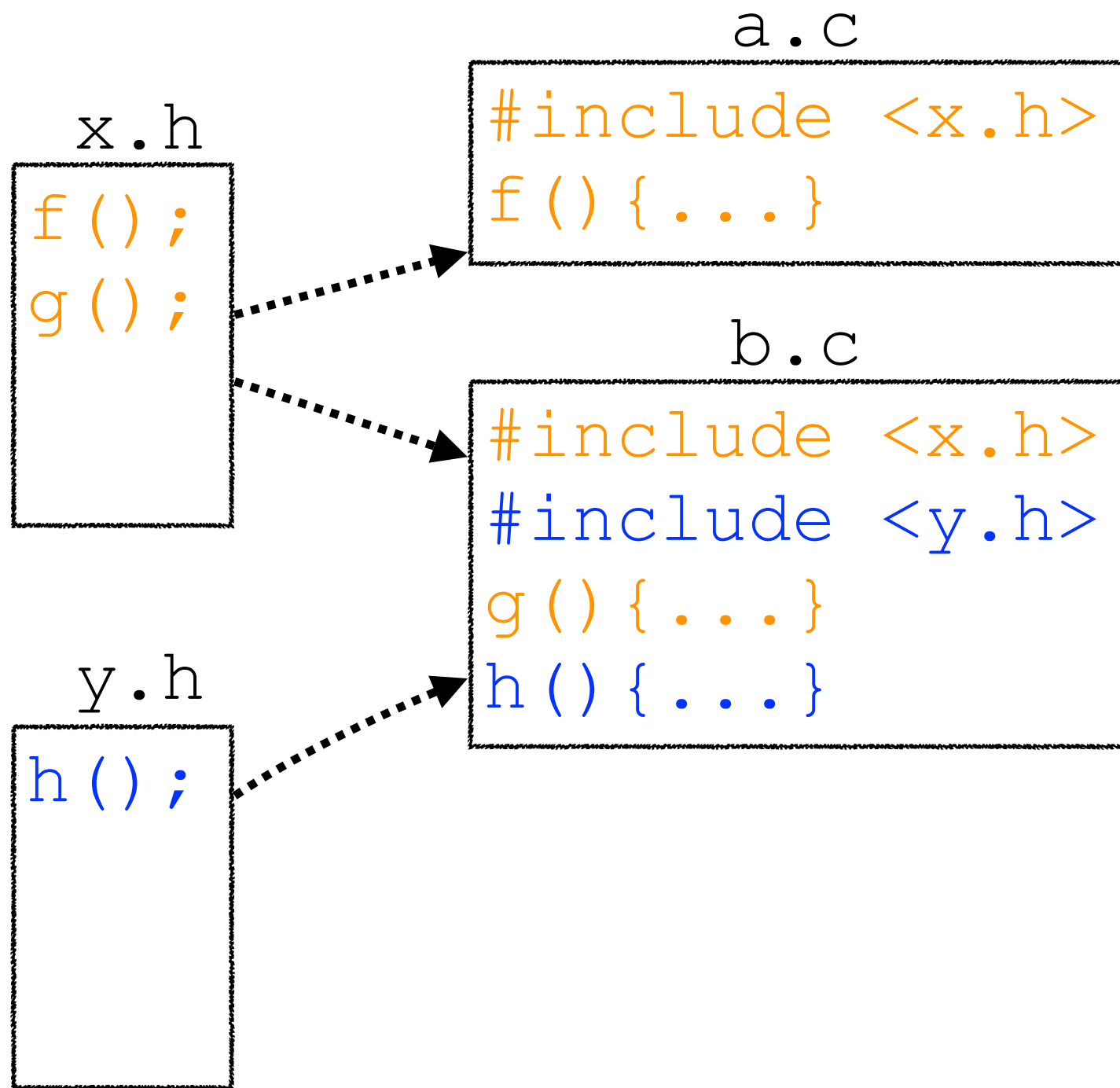
Programmstruktur



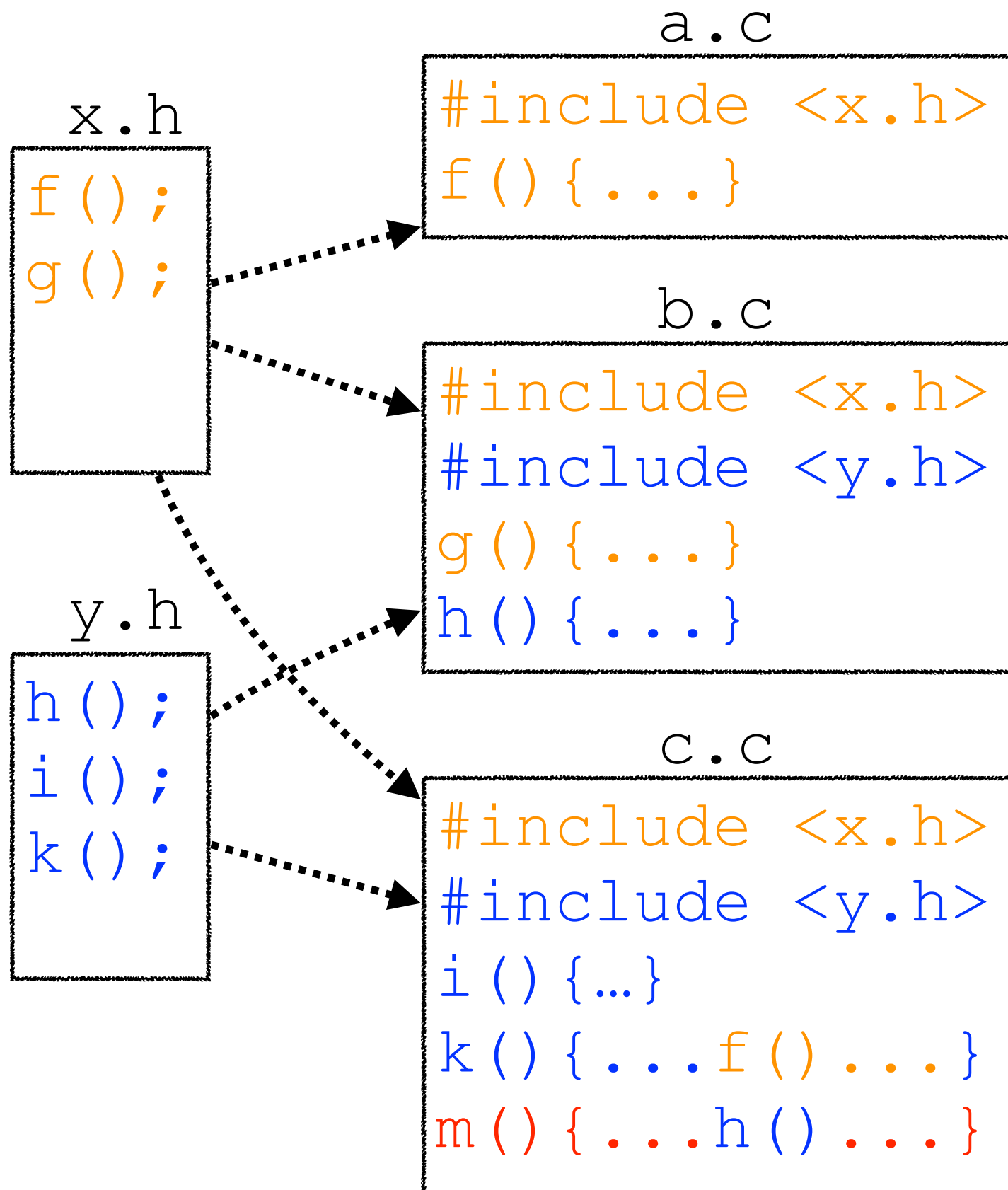
Programmstruktur



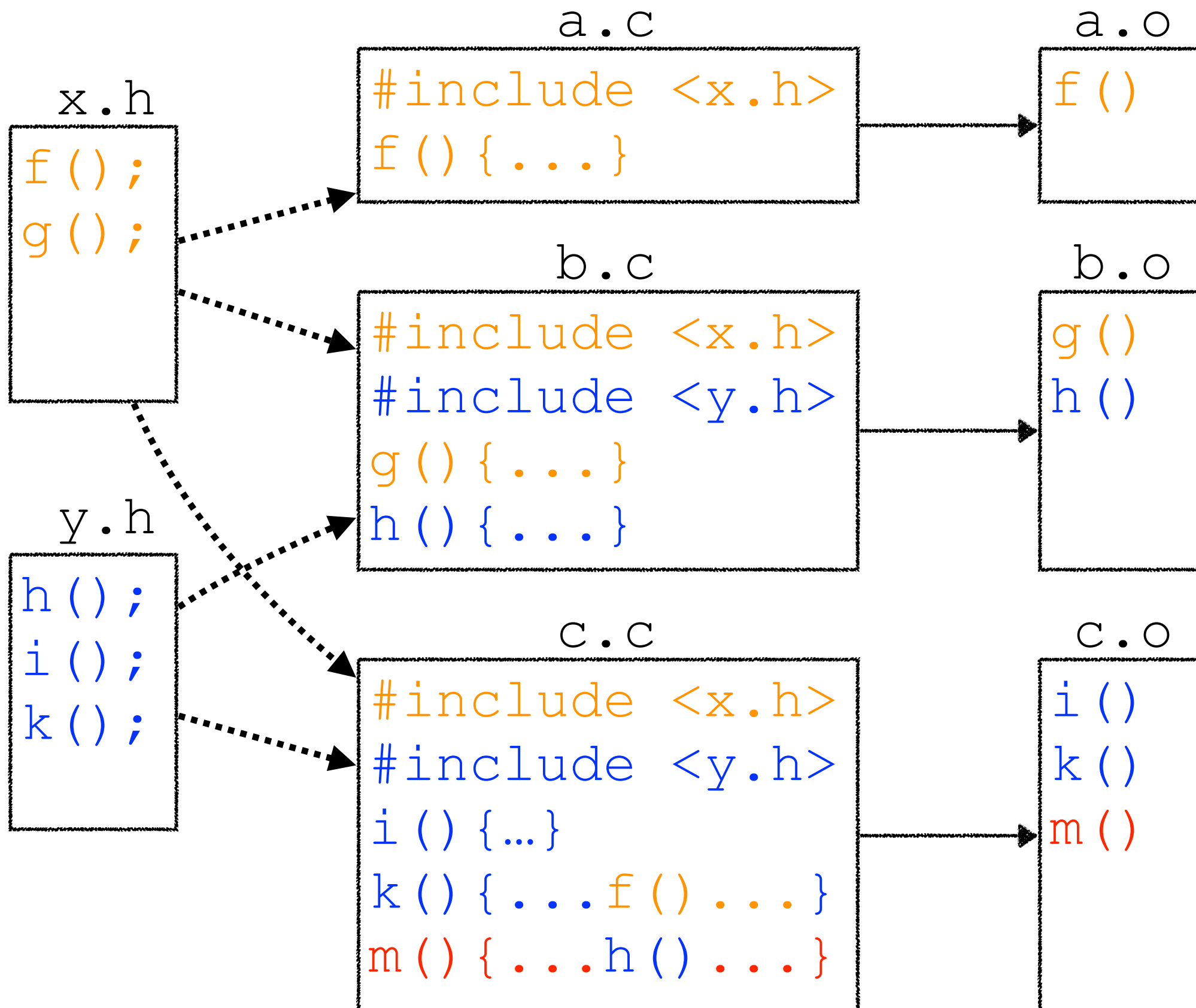
Programmstruktur



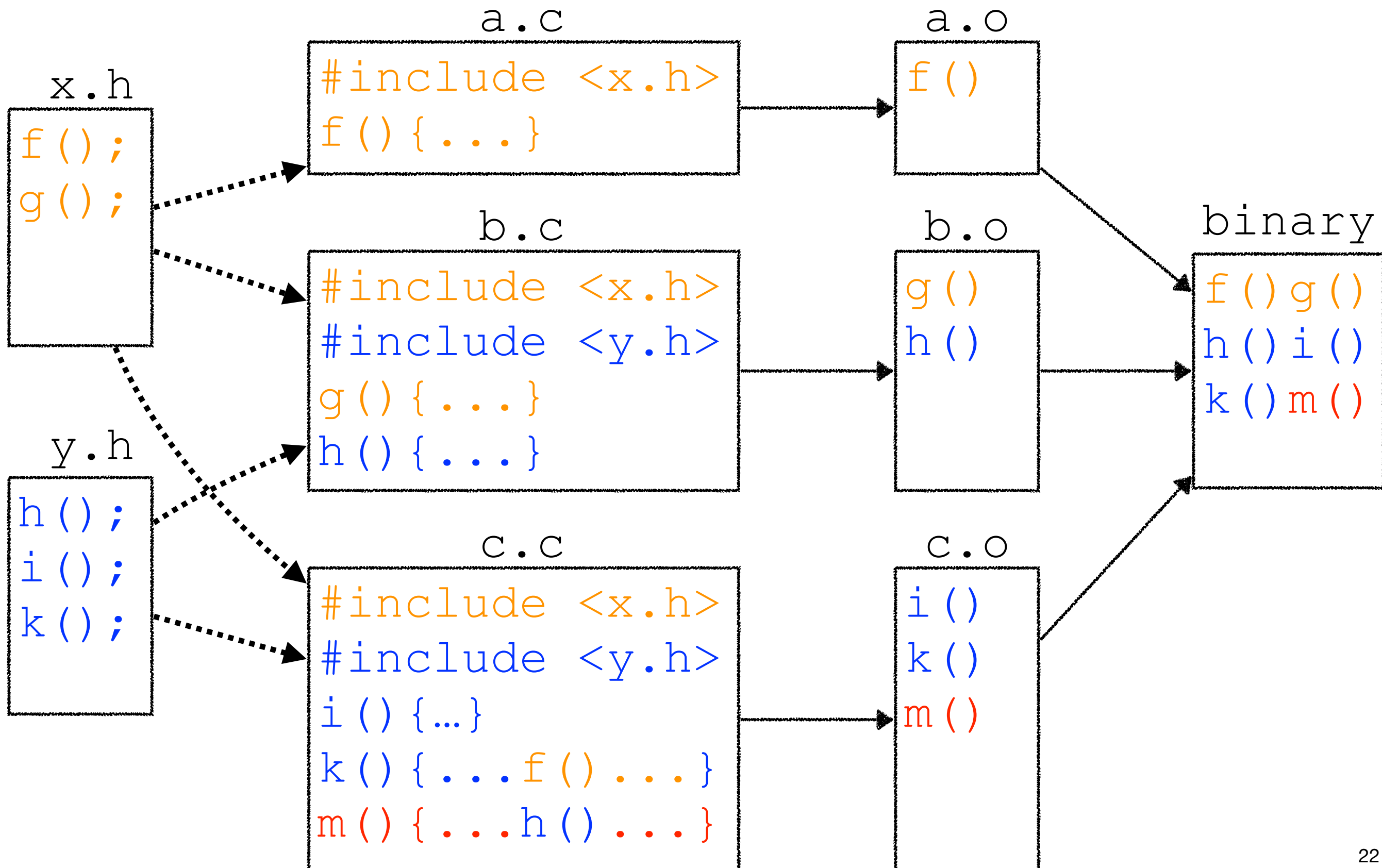
Programmstruktur



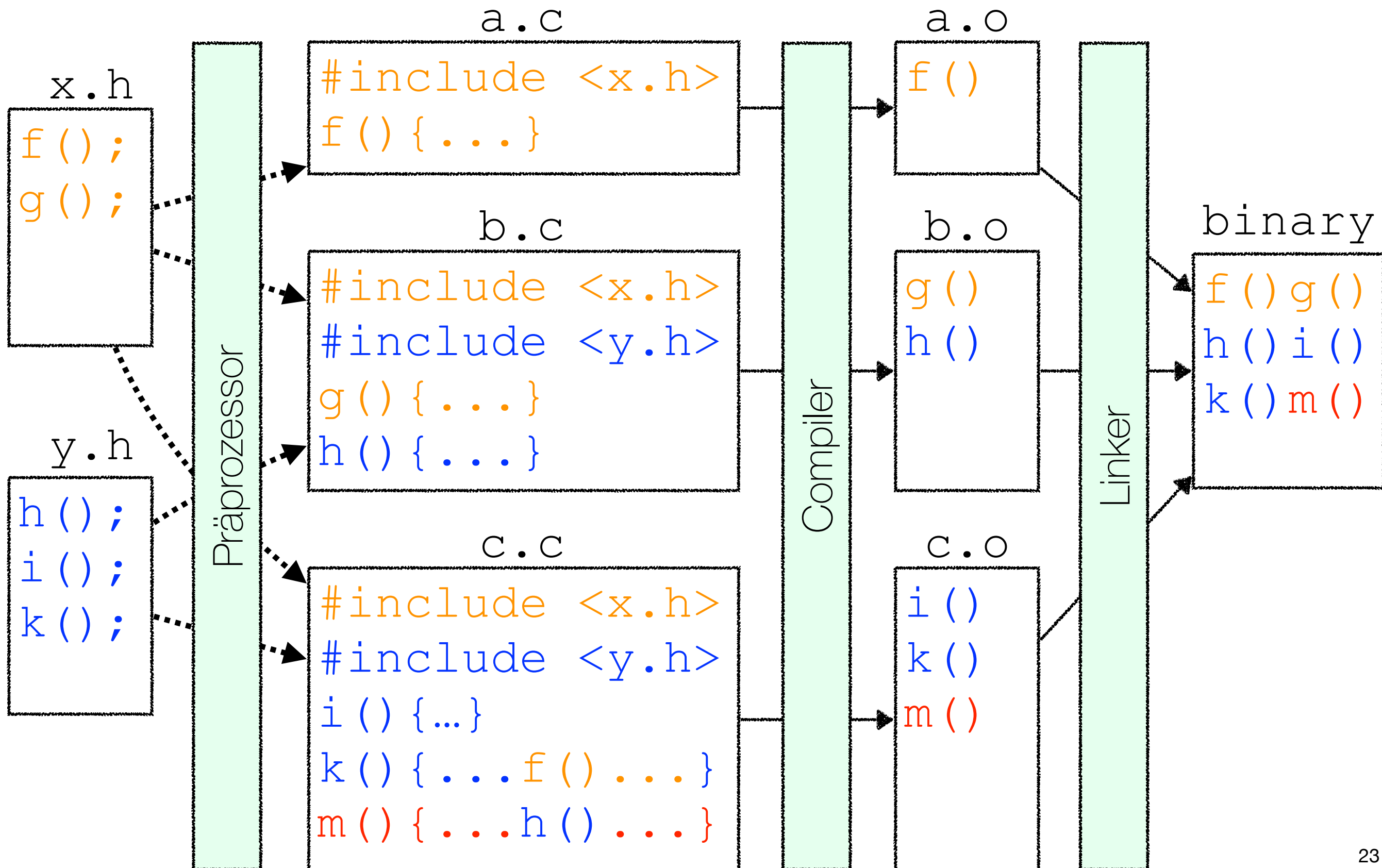
Programmstruktur



Programmstruktur



Programmstruktur



Beispiel: Fahrenheit to Celsius

```
#include <stdio.h>

int main() {
    typedef float Temperature;
    Temperature fahrenheit = 53;
    Temperature celsius;

    celsius = 5 * (fahrenheit - 32) / 9;
    printf("%f F is %f C\n", fahrenheit, celsius);

    return 0;
}
```

ftoc: **Header** Files

Temperature.h

```
#ifndef TEMPERATURE_H
#define TEMPERATURE_H

typedef float Temperature;

#endif
```



```
#include "Temperature.h"

Temperature toCelsius(Temperature fahrenheit);
```

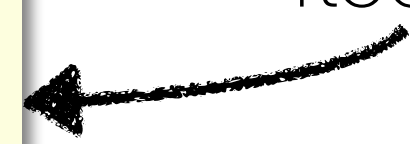
ftoc.h



```
#include "ftoc.h"

Temperature toCelsius(Temperature fahrenheit) {
    return 5.0 * (fahrenheit - 32.0) / 9.0;
}
```

ftoc.c



convert.c

ftoc: Präprozessor

```
#include <stdio.h>
#include "ftoc.h"
```

Präprozessor

```
int main() {
    Temperature fahrenheit = 53;
    Temperature celsius = toCelsius(fahrenheit);
    printf("%5.1f F is %5.1f C\n", fahrenheit, celsius);
    return 0;
}
```

```
#include <stdio.h>
#include "Temperature.h"
```

```
Temperature toCelsius(Temperature fahrenheit);
```

```
int main() {
    Temperature fahrenheit = 53;
    Temperature celsius = toCelsius(fahrenheit);
    printf("%5.1f F is %5.1f C\n", fahrenheit, celsius);
    return 0;
}
```


convert.c

ftoc: Präprozessor

```
#include <stdio.h>
#include "Temperature.h"

Temperature toCelsius(Temperature fahrenheit);

int main() {
    Temperature fahrenheit = 53;
    Temperature celsius = toCelsius(fahrenheit);
    printf("%5.1f F is %5.1f C\n", fahrenheit, celsius);
    return 0;
}
```

Präprozessor

```
#include <stdio.h>
#ifndef TEMPERATURE_H
#define TEMPERATURE_H
typedef float Temperature;
#endif
```

```
Temperature toCelsius(Temperature fahrenheit);

int main() {
    Temperature fahrenheit = 53;
    Temperature celsius = toCelsius(fahrenheit);
    printf("%5.1f F is %5.1f C\n", fahrenheit, celsius);
    return 0;
}
```

convert.c
nach
Präprozessor
-Durchlauf

Beispiel: Fahrenheit to Celsius



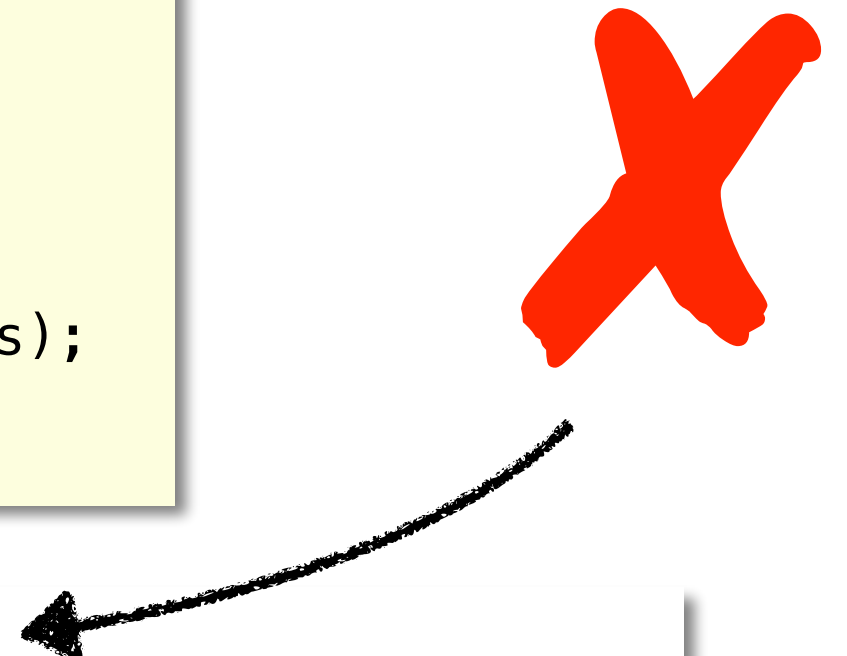
- **convert.c**
- **ftoc.c**
- **convert_baf.c**
- **ctof.c**

ftoc: **Compiler & Linker**

```
#include <stdio.h>
#ifdef TEMPERATURE_H
#define TEMPERATURE_H
typedef float Temperature;
#endif

Temperature toCelsius(Temperature fahrenheit);

int main() {
    Temperature fahrenheit = 53;
    Temperature celsius = toCelsius(fahrenheit);
    printf("%5.1f F is %5.1f C\n", fahrenheit, celsius);
    return 0;
}
```



```
esch:ftoc esch$ gcc convert.c -o convert
Undefined symbols for architecture x86_64:
  "_toCelsius", referenced from:
      _main in convert-a1affa.o
ld: symbol(s) not found for architecture x86_64
clang: error: linker command failed with exit code 1 (use
-v to see invocation)
```

ftoc: **Compiler & Linker**

```
#include <stdio.h>
#ifdef TEMPERATURE_H
#define TEMPERATURE_H
typedef float Temperature;
#endif

Temperature toCelsius(Temperature fahrenheit);

int main() {
    Temperature fahrenheit = 53;
    Temperature celsius = toCelsius(fahrenheit);
    printf("%5.1f F is %5.1f C\n", fahrenheit, celsius);
    return 0;
}
```

```
esch$ gcc -c convert.c
esch$ file convert.o
convert.o: Mach-O 64-bit object x86_64
esch$ gcc -c ftoc.c
esch$ file ftoc.o
ftoc.o: Mach-O 64-bit object x86_64
esch$ gcc convert.o ftoc.o -o convert
esch$ file convert
convert: Mach-O 64-bit executable x86_64
```

compile to
object file

link to
executable

ftoc: **Alternative Implementierung**

```
#include "ftoc.h"
```

```
Temperature toCelsius(Temperature fahrenheit) {  
    return 5.0 * (fahrenheit - 32.0) / 9.0;  
}
```

ftoc.c



```
#include "ftoc.h"
```

```
Temperature toCelsius(Temperature fahrenheit) {  
    return (Temperature)(5 * ((int)fahrenheit - 32) / 9);  
}
```

ftoc2.c



```
esch$ gcc convert.o ftoc.o -o convert
```

```
esch$ gcc convert.o ftoc2.o -o convert2
```



linken unterschiedlicher
Implementierungen

ftoc: Alternative Implementierung

```
#include "ftoc.h"
```

```
Temperature toCelsius(Temperature fahrenheit) {  
    return 5.0 * (fahrenheit - 32.0) / 9.0;  
}
```

ftoc.c



```
#include "ftoc.h"
```

```
Temperature toCelsius(Temperature fahrenheit) {  
    return (Temperature)(5 * ((int)fahrenheit - 32) / 9);  
}
```

ftoc2.c



```
esch:ftoc esch$ gcc convert.o ftoc.o ftoc2.o -o convert  
duplicate symbol _toCelsius in:
```

```
    ftoc.o
```

```
    ftoc2.o
```

```
ld: 1 duplicate symbol for architecture x86_64
```

```
clang: error: linker command failed with exit code 1
```

```
(use -v to see invocation)
```



ftoc & ctof

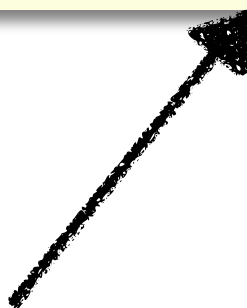
```
#include <stdio.h>
#include "ftoc.h"
#include "ctof.h"

int main() {
    Temperature f = 53;
    Temperature c;

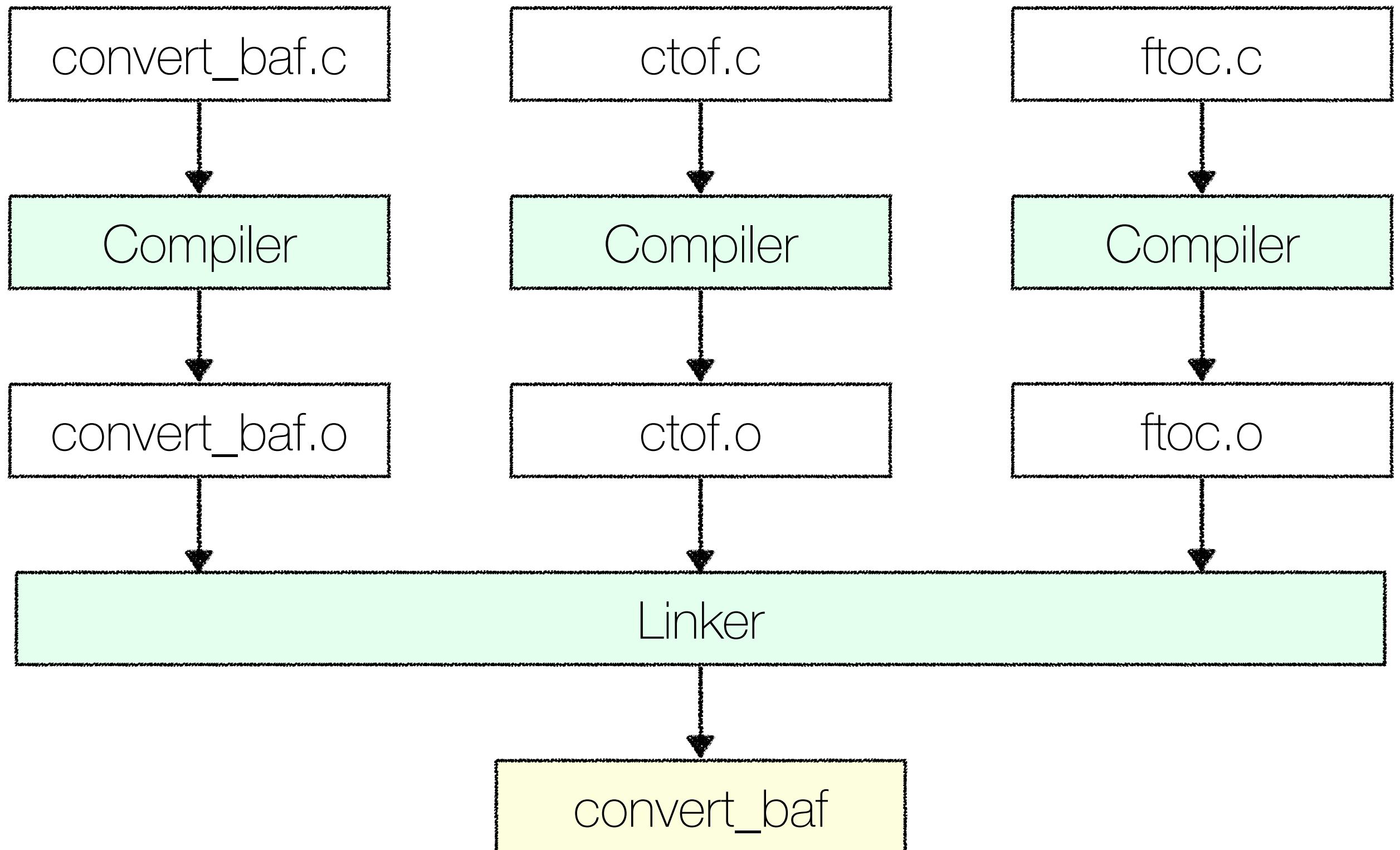
    c = toCelsius(f);
    Temperature fBack = toFahrenheit(c);

    printf("%5.1f F is %5.1f C and %5.1f in F\n", f, c, fBack);
    return 0;
}
```

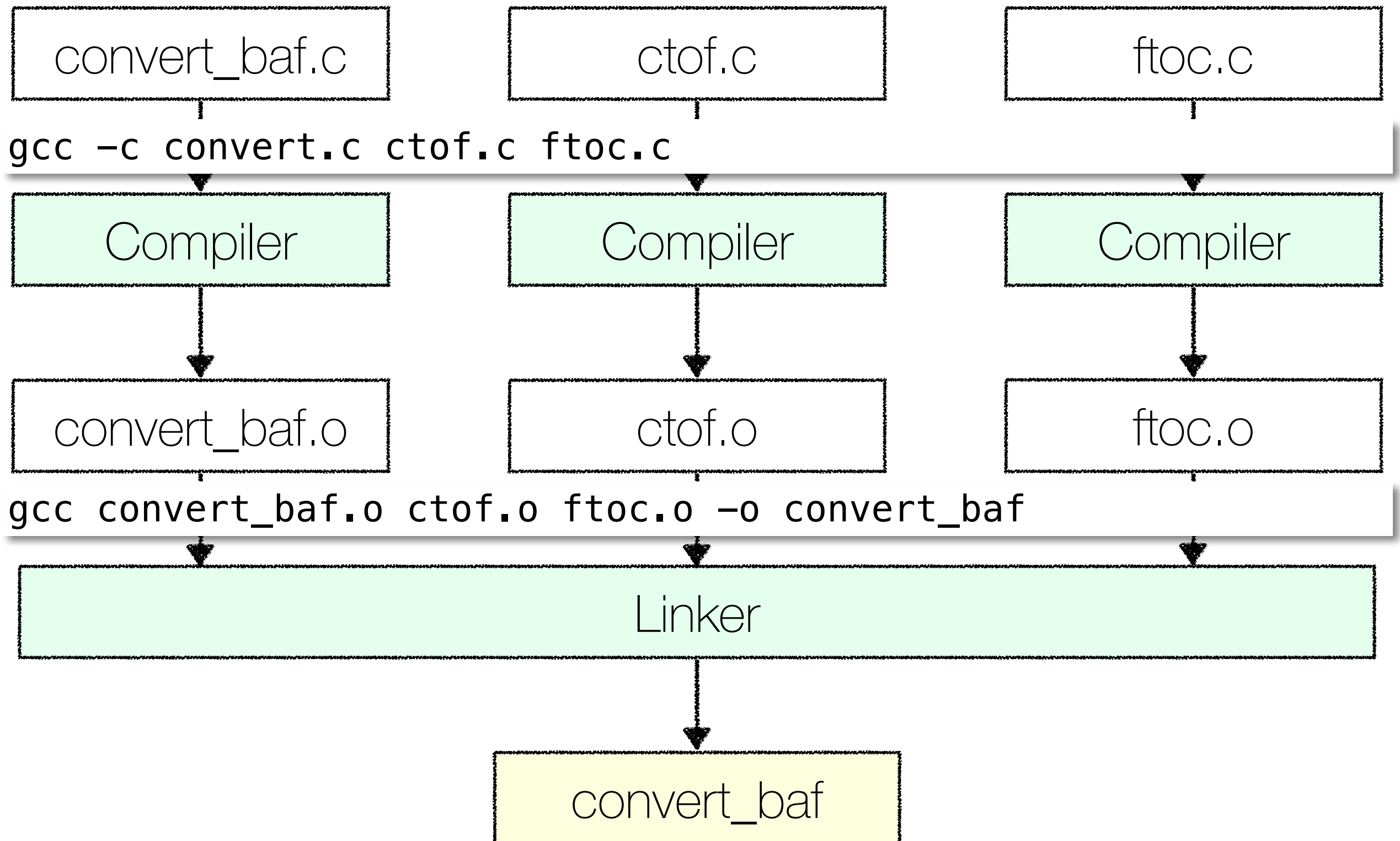
convert_baf.c



Compiler & Linker



Compiler & Linker



#ifndef Guards

```
#ifndef TEMPERATURE_H  
#define TEMPERATURE_H  
typedef float Temperature;  
#endif
```

Temperature.h



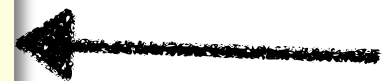
```
#include "Temperature.h"  
Temperature toCelsius(Temperature fahrenheit);
```

ftoc.h



```
#include "ftoc.h"  
#include "Temperature.h"  
Temperature toCelsius(Temperature fahrenheit) {  
    return 5.0 * (fahrenheit - 32.0) / 9.0;  
}
```

ftoc.c



#ifdef Guards

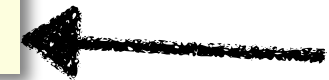
```
#ifndef TEMPERATURE_H  
#define TEMPERATURE_H  
typedef float Temperature;  
#endif
```

Temperature.h



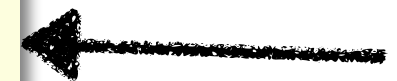
```
#include "Temperature.h"  
Temperature toCelsius(Temperature fahrenheit);
```

ftoc.h



```
#include "ftoc.h"  
#include "Temperature.h"  
Temperature toCelsius(Temperature fahrenheit) {  
    return 5.0 * (fahrenheit - 32.0) / 9.0;  
}
```

ftoc.c



```
Lagavulin:ftoc_include_failure esch$ gcc -c -std=c99 ftoc.c
```

```
In file included from ftoc.c:2:
```

```
./Temperature.h:1:15: warning: redefinition of typedef 'Temperature' is a C11 feature  
    [-Wtypedef-redefinition]  
typedef float Temperature;
```

```
^
```

```
./ftoc.h:1:10: note: './Temperature.h' included multiple times, consider augmenting this header  
    with #ifdef guards
```

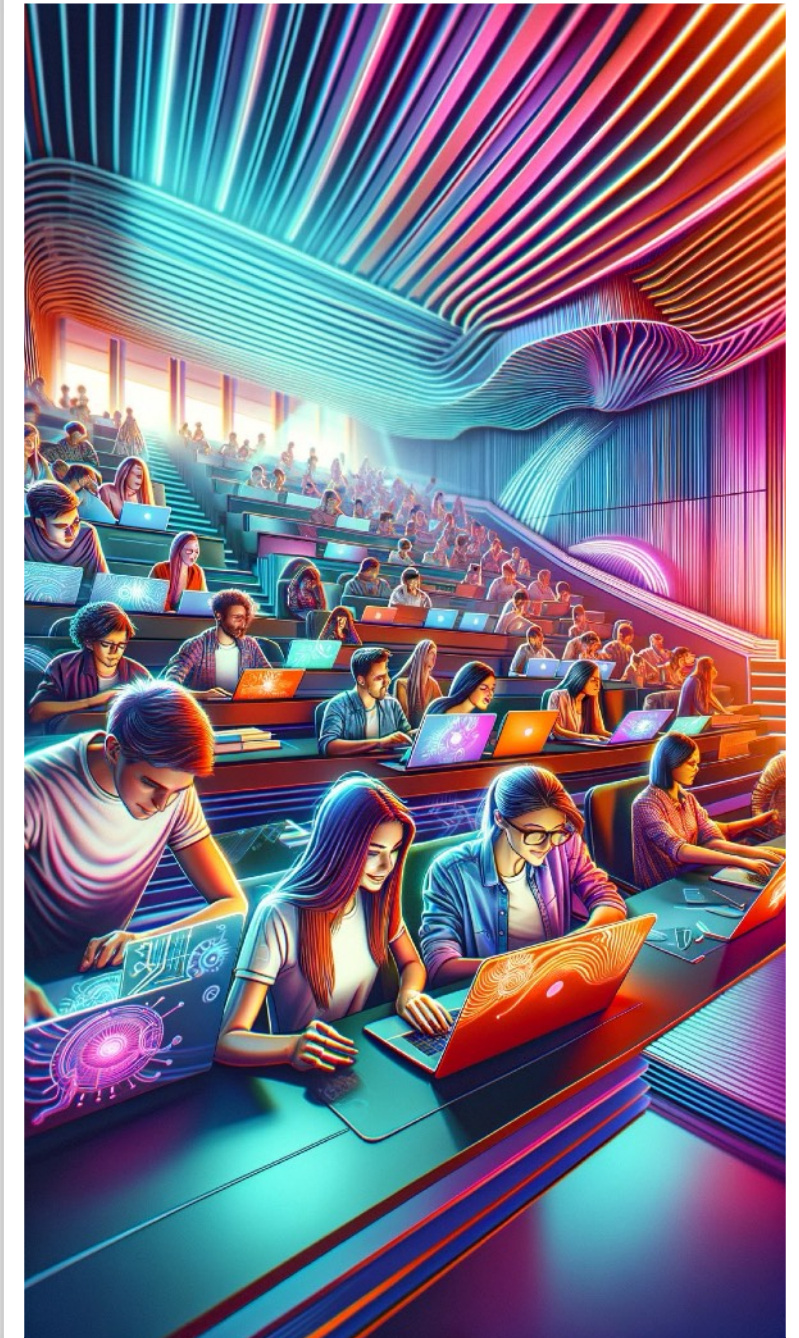
```
#include "Temperature.h"
```

```
^
```

```
1 warning generated.
```

Übung

- ▶ In Moodle finden Sie die Datei `math.c`
- ▶ Deklarieren Sie die dort aufgerufenen mathematischen Funktionen (`add`, `subtract`, ...) in der Datei `operations.h`
- ▶ Implementieren Sie die Funktionen in einer Datei `operations.c`
- ▶ Kompilieren und linken Sie das Programm, um es zu testen.



Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was macht der Präprozessor und welche Präprozessor-Direktiven gibt es?
- Was macht der Linker?
- Wie arbeiten Compiler und Linker zusammen?

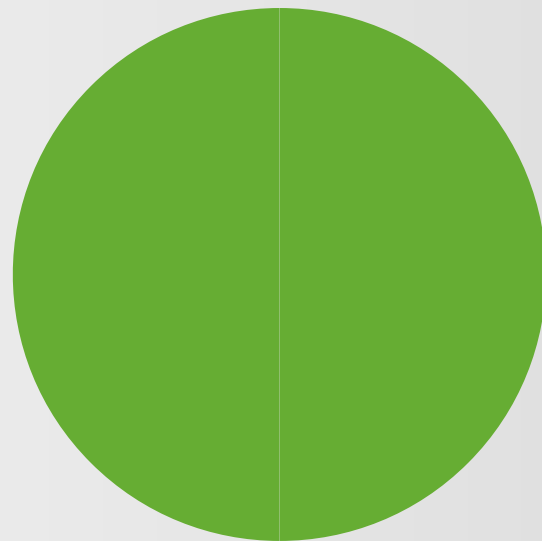


45 Sekunden

Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was macht der Präprozessor und welche Präprozessor-Direktiven gibt es?
- Was macht der Linker?
- Wie arbeiten Compiler und Linker zusammen?



45 Sekunden

Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was macht der Präprozessor und welche Präprozessor-Direktiven gibt es?
- Was macht der Linker?
- Wie arbeiten Compiler und Linker zusammen?

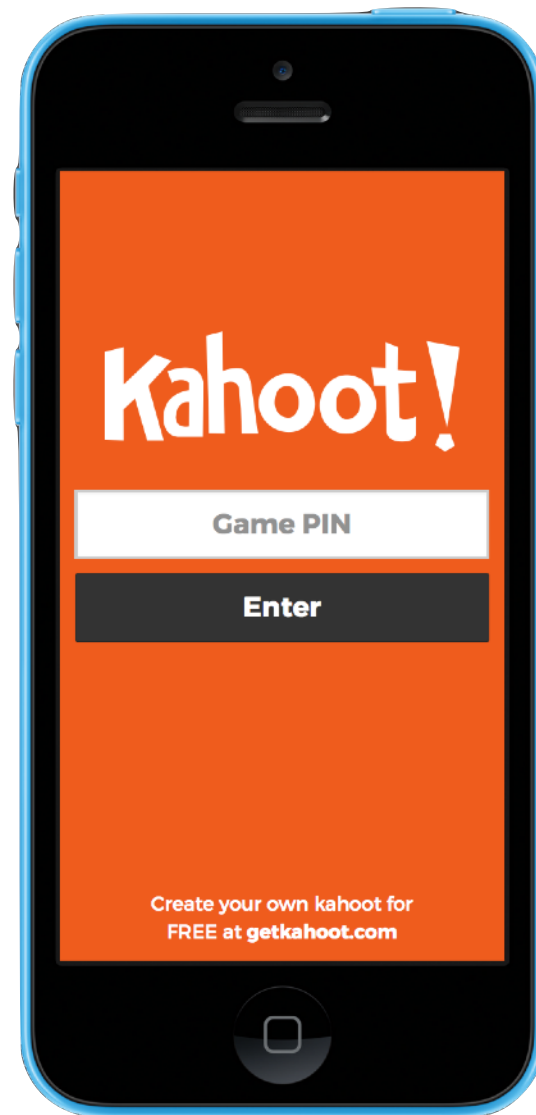


45 Sekunden

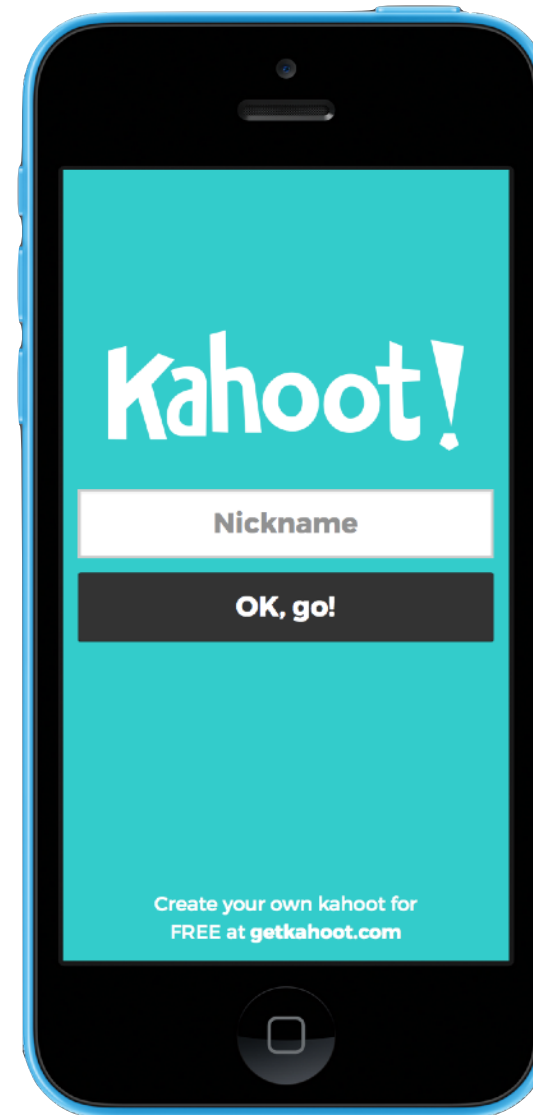
Kahoot!

go to
kahoot.it

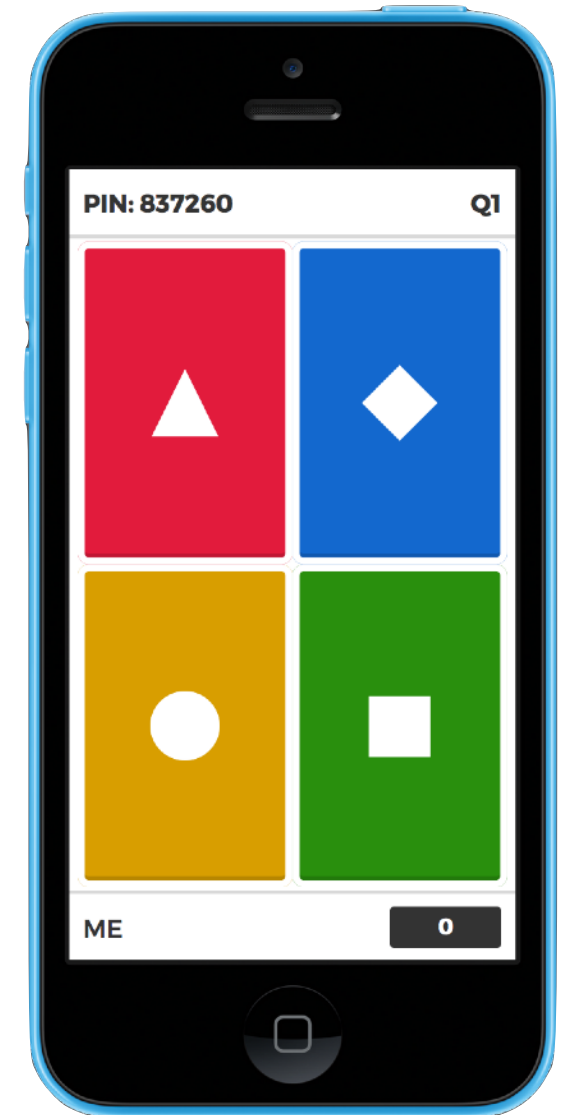
PIN
eingeben



Nickname
eingeben



mitspielen



1

2

3

4

